

Enginyeria del Programari de Components i Sistemes Distribuïts

SEGONA PROVA PRÀCTICA – PRAC 2

Presentació

A la PRAC 1 s'ha dissenyat una solució per al cas **Photo&Film4You** basada en microserveis, emprant models de representació i patrons de disseny. En aquesta PRAC 2 es demana implementar un subconjunt d'aquest disseny, a partir de desenvolupar tres microserveis considerant diferents aspectes com: estat propi, comunicació síncrona, lògica de negoci, consistència de les dades i desplegament en local.

Per elaborar aquesta pràctica es formulen dos exercicis a partir del cas **Photo&Film4You**:

1. Implementar tres dels microserveis dissenyats a la PRAC1, segons un conjunt de consideracions tècniques i la documentació de les evidències.
2. Raonar com resoldre la gestió de transaccions en una arquitectura distribuïda de microserveis.

La PRAC 2 abasta els continguts del llibre "Microservices Patterns" (veure en l'apartat "Recursos" de l'assignatura els capítols que cal estudiar).

Competències

En aquesta PRAC 2 es treballa les següents competències del Grau en Enginyeria Informàtica:

- Saber construir aplicacions informàtiques mitjançant tècniques de desenvolupament, integració i reutilització.
- Saber proposar i avaluar diferents alternatives per resoldre un problema concret.

Objectius

Els objectius de la PRAC 2 són:

- Implementar microserveis centrats en la lògica de negoci i l'accés a dades.
- Desplegar microserveis amb estat propi i interoperabilitat síncrona o asíncrona.
- Aplicar conceptes bàsics de l'arquitectura de microserveis.
- Dominar les tecnologies bàsiques necessàries per a la implementació de microserveis.

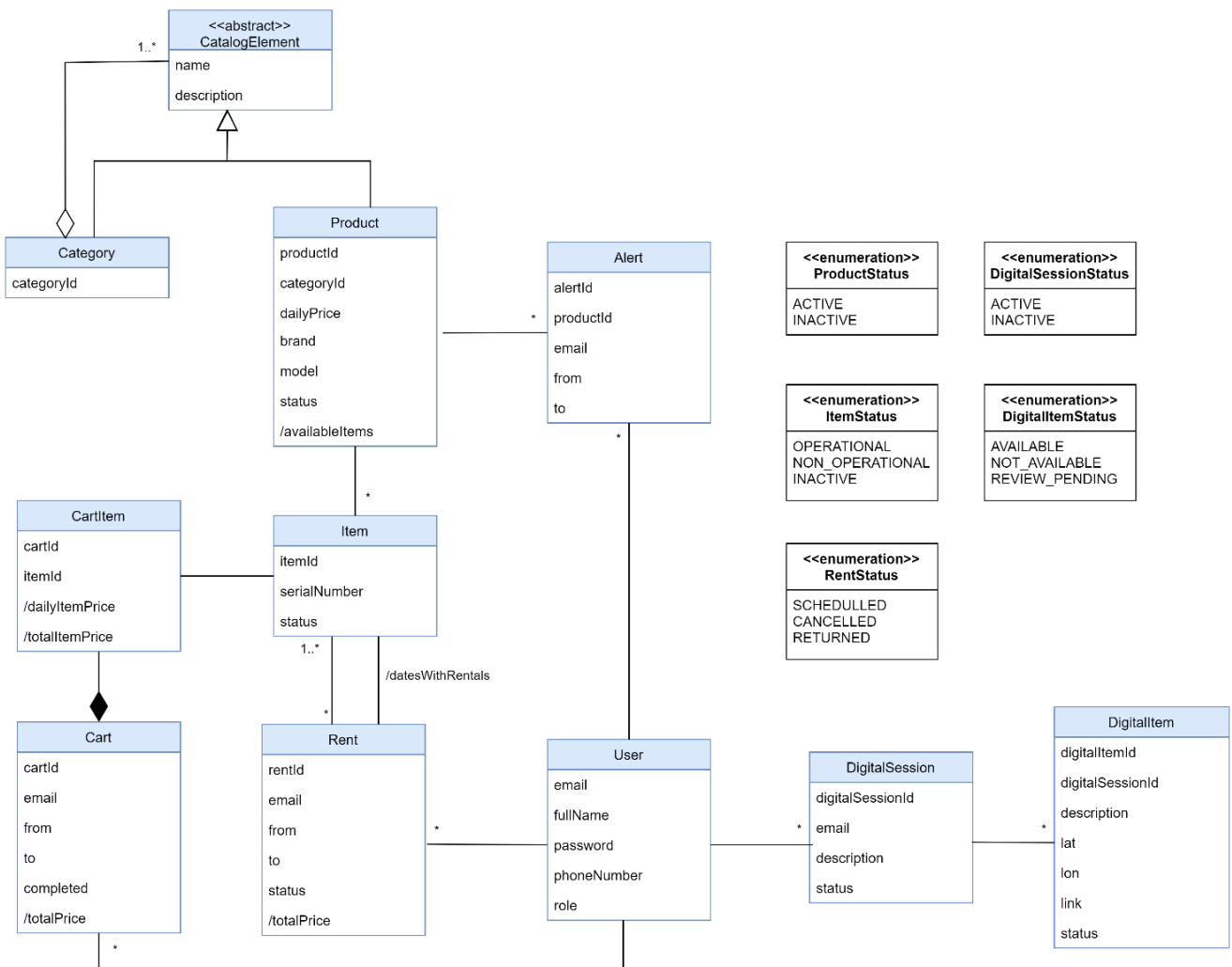
Descripció de la PRAC 2

Exercici 1 (8 punts)

L'exercici 1 consta de dues parts, la primera part consistirà en la implementació d'un programari, i la segona part en la documentació de les evidències del que s'ha implementat. A continuació, es descriuen les tasques concretes a realitzar en cadascuna de les dues parts.

Primera part (6 punts)

Implementar un subconjunt dels microserveis dissenyats a la PRAC1. S'ha de basar en la solució publicada per la PRAC anterior i per tant es parteix del següent diagrama del model general del domini, del qual es poden extreure les principals entitats (amb els seus atributs) i relacions:



El subconjunt de microserveis a implementar és el format per *Product Catalog*, *User* i *Notification*:

- Amb el propòsit de simplificar i facilitar la implementació, el microservei *Digital Service* s'implementarà dins del microservei *User Service*.
- El microservei *Product Catalog* servirà tant per donar cobertura a les diverses operacions que els administradors de la plataforma necessiten per gestionar l'estoc (crear/eliminar seccions/subseccions, productes i unitats de producte) com per a les consultes del catàleg de productes que realitzaran els usuaris.
- El microservei *User* servirà per gestionar les altes, les dades personals dels usuaris, la configuració d'alertes, els continguts digitals i els seus elements. Els usuaris han de poder donar d'alta alertes de disponibilitat per un producte i un rang de dates concrets. Quan una unitat d'aquest producte quedi lliure per a aquest rang de dates, cal emetre un missatge a la cua perquè un altre servei s'encarregui de fer efectiva l'alerta (microservei *Notification*). També cal donar d'alta alertes de revisió d'un element d'un contingut digital.
- El microservei *Notification* té la finalitat d'emetre notificacions als usuaris interessats en un producte concret, quan s'afegeixen unitats o alguna queda lliure per ser llogada, o també per notificar una revisió respecte un contingut digital. Aquest microservei ha d'escollar la cua de missatges de manera que quan s'alliberi una unitat de producte per a una data configurada en una alerta, al corresponent usuari se li envii un correu electrònic amb l'avís. També als administradors quan un element digital ha de ser revisat.

D'acord amb el descrit, aquests tres microserveis tindran com a mínim les següents operacions:

Microservei *Product Catalog* (catàleg de productes):

- crear secció/subsecció
- crear producte
- crear unitat de producte
- eliminar producte
- canviar l'estat d'una unitat de producte a operatiu/no-operatiu
- consulta de seccions/subseccions per nom
- consulta de seccions/subseccions per descripció
- consulta de seccions/subseccions per secció "pare"
- consulta de productes per nom o per secció
- consulta del detall d'un producte
- consulta del detall d'una unitat

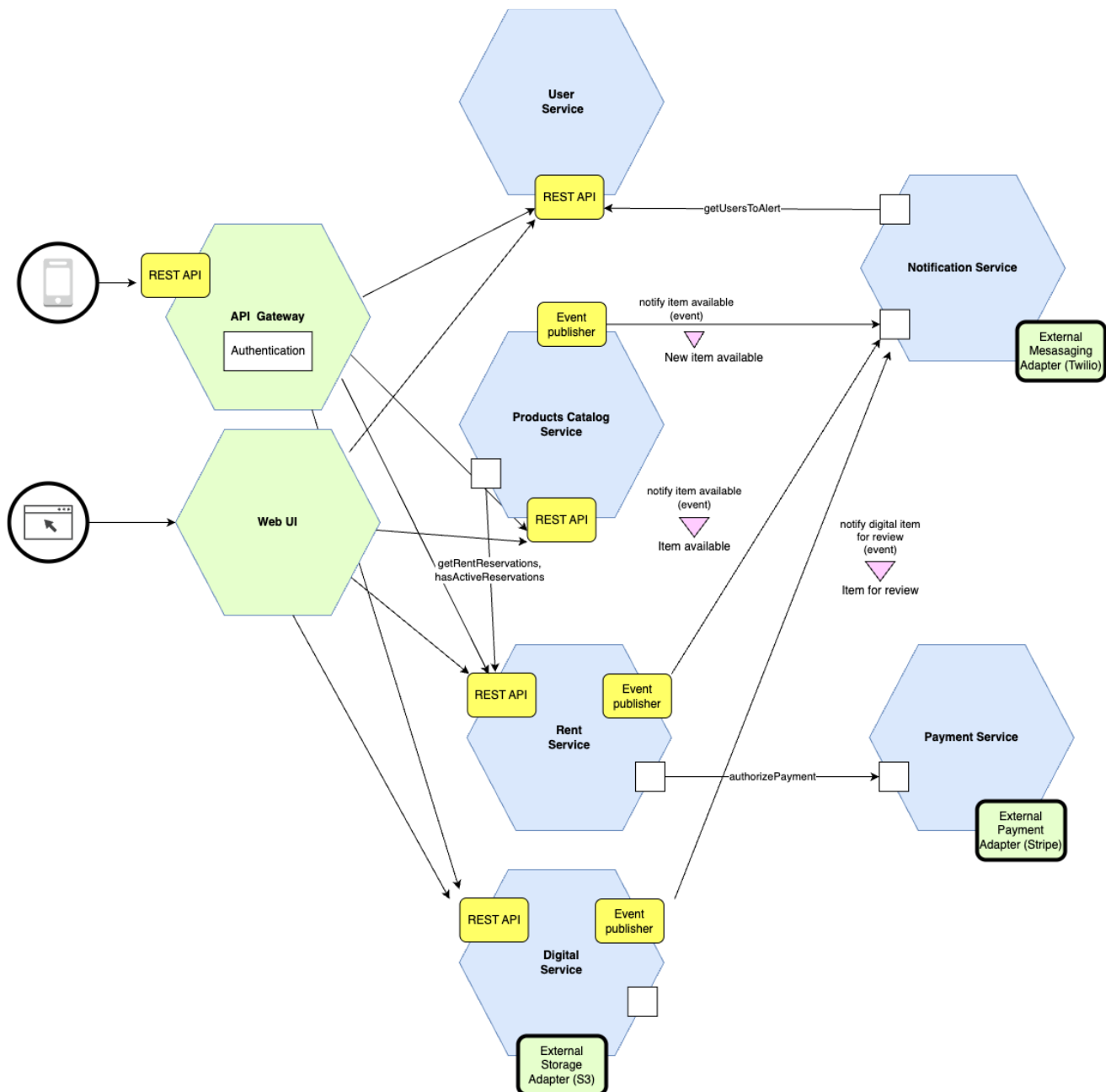
Microservei *User* (gestió d'usuaris):

- alta d'usuari
- alta d'alerta de disponibilitat
- consulta de les alertes d'un usuari per a un interval concret de dates
- consulta de les alertes per producte i data
- consulta d'usuaris a alertar per a un producte i data (operació *getUsersToAlert*)
- consulta del detall d'una alerta
- alta, baixa o modificació d'una sessió digital
- alta, baixa o modificació d'un contingut digital per una sessió
- consulta de les sessions digitals per un usuari
- consulta del detall d'una sessió digital
- consulta dels continguts d'una sessió digital
- marcar un contingut digital com pendent de revisió
- marcar un contingut digital com retirat per problema de propietat intel·lectual
- marcar un contingut digital prèviament retirat com aprovat de nou.

Microservei *Notification* (notificacions automàtiques a usuaris):

- enviar notificació (operació de sistema a partir d'un esdeveniment asíncron, no disponible per als usuaris)

En el següent diagrama es pot veure la col·laboració entre els diversos serveis del sistema.



Consideracions prèvies

Per simplificar i facilitar la implementació de la PRAC2, s'han unificat els microserveis *Digital* i *User* en aquest segon, respecte la solució de PRAC1.

Per aconseguir la flexibilitat, resiliència i rendiment que caracteritza aquesta arquitectura, cal complir una sèrie de requeriments:

- Cada microservei s'ha de poder executar independentment, tenir estat propi (i accedir de manera independent a la BBDD), implementar només la seva lògica de negoci, exposar el seu contracte (definició de l'API), i gestionar el seu propi *log*.
- Per certes operacions, pot ser necessària la col·laboració entre diversos microserveis per obtenir informació. En aquest cas, els microserveis poden interoperar entre ells emprant crides REST.

Coses que queden fora de l'abast del projecte en la fase pilot que es demana implementar:

- Autenticació: no serà necessari cap tipus d'autenticació per a cap de les operacions, es dona per fet que aquesta part de la funcionalitat del sistema la proporciona l'API Gateway (que no s'ha d'implementar).
- Service Discovery i Registration: no s'ha d'implementar cap tipus de funcionalitat de registre i recerca de serveis, es dona per fet que només existeix una única instància, en una ubicació coneguda.
- Lliurament de correus: per no haver d'implementar aquesta part, de poc valor didàctic dins la PRAC, us suggerim simular el comportament amb l'ús de *Mocks*. Cal presentar almenys una línia de log cada vegada que s'envii una notificació (per exemple, mostrar mitjançant el log del microservei *Notification* una línia amb nivell INFO que digui "S'ha notificat per correu l'usuari XXX YYY ZZZ que existeix una unitat disponible del producte AAA").

Plataforma de desenvolupament

- Entorn de desenvolupament: IntelliJ IDEA / Eclipse
- JDK: OpenJDK 11
- Framework: Spring / Spring Boot
- Gestió de dependències i build: Maven
- Servidor HTTP: Tomcat (integrat per defecte amb Spring Boot)
- Llibreria crides REST: Jersey
- Comunicació asíncrona entre serveis: Apache Kafka
- BBDD: PostgreSQL
- Implementació ORM: Hibernate
- Generació de la documentació de l'API OpenAPI v3: springdoc-openapi
- Log: Log4j2
- Mocks: Mockneat (per generar dades pseudo-aleatòries)

Punt de partida i recursos

Per tenir un punt de partida comú i homogeni, i centrar els esforços en el desenvolupament dels microserveis, juntament amb aquest enunciat es proporciona un enllaç a un repositori públic (<https://github.com/UOC-EPCSD-SPRING-2025>) amb els següents recursos:

- Un arxiu *docker-compose.yml* per instal·lar la infraestructura bàsica de la implementació d'aquest exercici, i un arxiu *README.md* amb instruccions per arrencar-la.
- Esquelets d'aplicació Spring Boot amb les funcionalitats bàsiques per generar:
 - un arxiu java jar executable amb totes les dependències contingudes en el projecte (per fer debug localment);
 - un arxiu d'imatge docker per cada microservei i totes les seves dependències (per poder fer l'entrega).
- Un conjunt d'enllaços a documentació, guies i tutorials relatius tant a les eines a utilitzar com a la pròpia implementació dels microserveis sol·licitats (es penjaran a l'aula de Laboratori).

Cadascun dels esquelets d'aplicació que es proporcionen són un projecte Spring Boot complet preconfigurat amb les dependències mínimes necessàries per fer tota la implementació que es demana en aquest enunciat. Això no significa que no es puguin incloure altres dependències si us sembla adequat, però amb el que n'hi ha és suficient per fer tota la implementació.

Amb els contenidors de l'arxiu *docker-compose* arrencats, tots els esquelets de microservei haurien de posar-se en marxa i connectar a la infraestructura tal com es lliuren amb aquesta PRAC2. A més, aprofitant que els esquelets porten el seu propi servidor HTTP encastat, s'ha inclòs un mòdul *springdoc-openapi* que auto-genera i publica l'arxiu de definició del microservei (API) en format *JSON/OpenAPI v3*, així com una petita interfície web *SwaggerUI* que permet fer crides a les operacions dels microserveis.

Les adreces per accedir a aquestes funcionalitats són:

- arxius de definició dels serveis:
 - *Product Catalog*: <http://localhost:18081/v3/api-docs/>
 - *User*: <http://localhost:18082/v3/api-docs/>
- interfícies web SwaggerUI per a proves:
 - *Product Catalog*: <http://localhost:18081/swagger-ui/index.html>
 - *User*: <http://localhost:18082/swagger-ui/index.html>

Com es pot veure, aquests exemples formen una base força completa des d'on començar el desenvolupament. Per tant, s'hauria d'evitar fer modificacions en els seus components bàsics (excepte petites configuracions, etc.), i només afegir les funcionalitats que es demanen en aquest enunciat. Trobareu més informació i suport tècnic al Laboratori de l'assignatura.

Lliuraments

Especificacions tècniques de l'aplicació a lliurar

El lliurament d'aquesta part de l'exercici serà un únic arxiu ZIP que ha d'incloure:

- Tot el codi i arxius de configuració del projecte en **un arxiu ZIP** (dins del ZIP "principal").
- Un **arxiu JAR** executable (generat pel propi esquelet del projecte lliurat) per cadascun dels serveis, que contindrà totes les classes i llibreries necessàries (dependències) per executar el servei en qüestió. Aquest arxiu es pot generar a partir de l'esquelet d'aplicació proporcionat amb una execució maven i el paquet "goal".

Per a més informació, podeu revisar el capítol “*How to generate the .jar*” en la pàgina 113 del tutorial d'instal·lació lliurat en el Laboratori.

- Un arxiu **readme.txt** amb la informació següent:
 - instruccions per posar en marxa la infraestructura necessària per a la plataforma (en cas que sigui diferent a la que es proporciona amb docker compose);
 - qualsevol configuració necessària per poder executar els serveis;
 - la comanda concreta per executar cada servei per separat (si no s'han realitzat canvis, n'hi hauria prou amb "java -jar productcatalog-VERSION.jar", etc.)

Solució

Es pot accedir a una implementació de referència dels microserveis especificats des dels repositoris públics GitHub localitzats a les adreces:

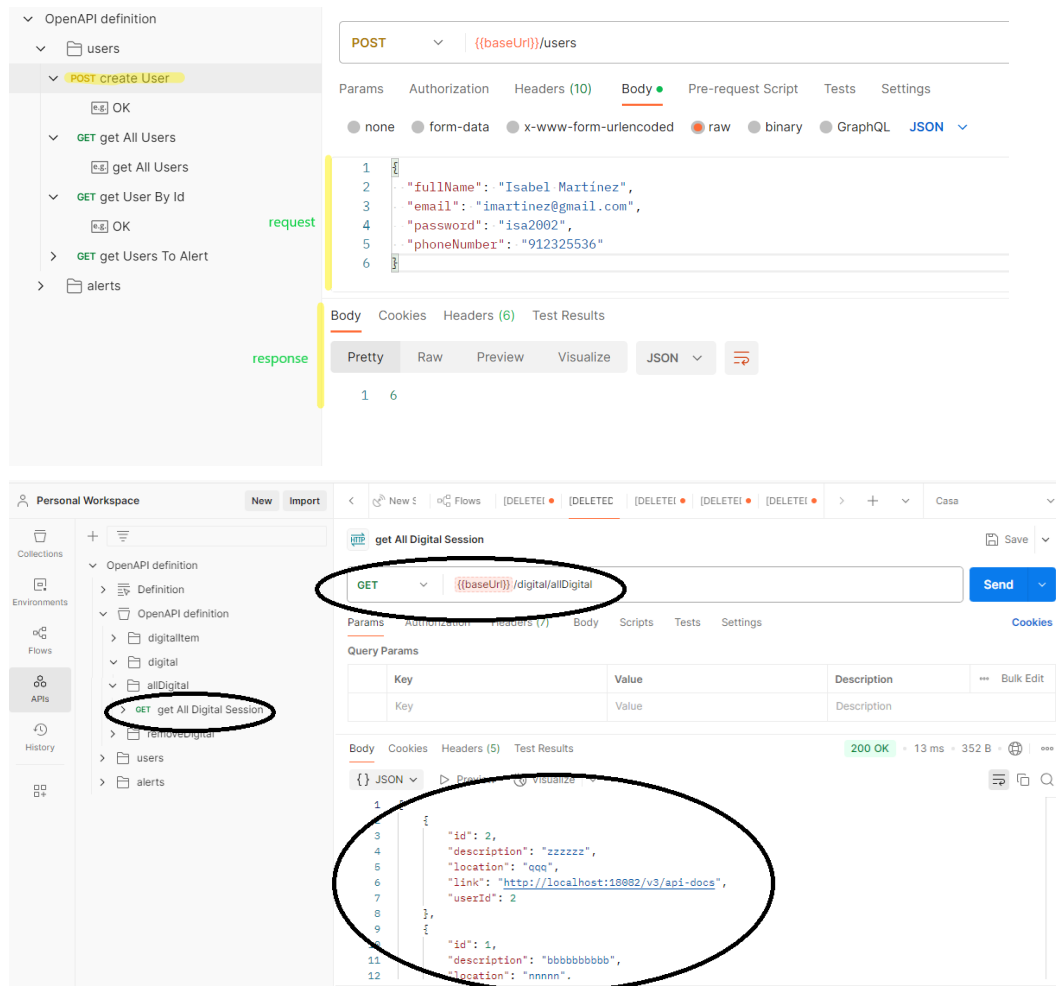
- <https://github.com/UOC-EPCSD-SPRING-2025/spring-2025-productcatalog-solution>
- <https://github.com/UOC-EPCSD-SPRING-2025/spring-2025-user-solution>
- <https://github.com/UOC-EPCSD-SPRING-2025/spring-2025-notification-solution>

Segona part (2 punts)

Per demostrar que s'han assolit els objectius de la implementació, haureu de proporcionar evidències del correcte funcionament dels microserveis. Per elaborar aquestes evidències us proposem utilitzar l'aplicació **Postman**, una eina que us permetrà importar l'arxiu de definició del servei (accessible des de la ruta `v3/api-docs` de cada microservei) i generar una col·lecció que contindrà totes les operacions disponibles. Com a mínim es realitzarà un exemple de cada tipus de crida amb dades "reals" un cop posats en marxa els microserveis.

Lliuraments

El lliurament d'aquesta part de l'exercici serà una captura de pantalla amb la crida i el resultat per cadascuna de les operacions implementades en la primera part.



The top screenshot shows a REST client interface. On the left, a sidebar lists API endpoints under 'OpenAPI definition' and 'users'. The main area shows a POST request to '({baseUri})/users'. The 'Body' tab is selected, showing a JSON payload:

```
{  "fullName": "Isabel Martinez",  "email": "imartinez@gmail.com",  "password": "isa2002",  "phoneNumber": "912325536"}
```

. The 'response' tab is also visible, showing a status of 200 OK.

The bottom screenshot shows a REST client interface. On the left, a sidebar lists API endpoints under 'OpenAPI definition' and 'digital'. The main area shows a GET request to '({baseUri})/digital/allDigital'. The 'Body' tab is selected, showing a JSON response:

```
{  "id": 2,  "description": "rzzzzz",  "location": "eeq",  "link": "http://localhost:18082/v3/api-docs",  "userId": 2},  {  "id": 1,  "description": "bbbbbbbb",  "location": "nnnnn"}
```

. The response status is 200 OK.

Solució

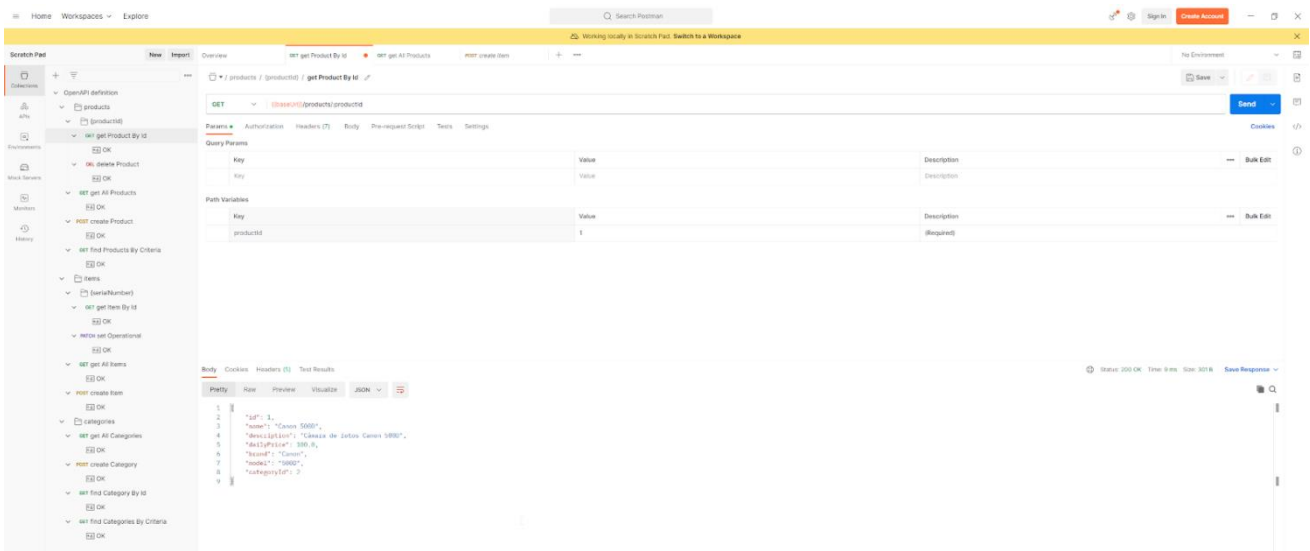
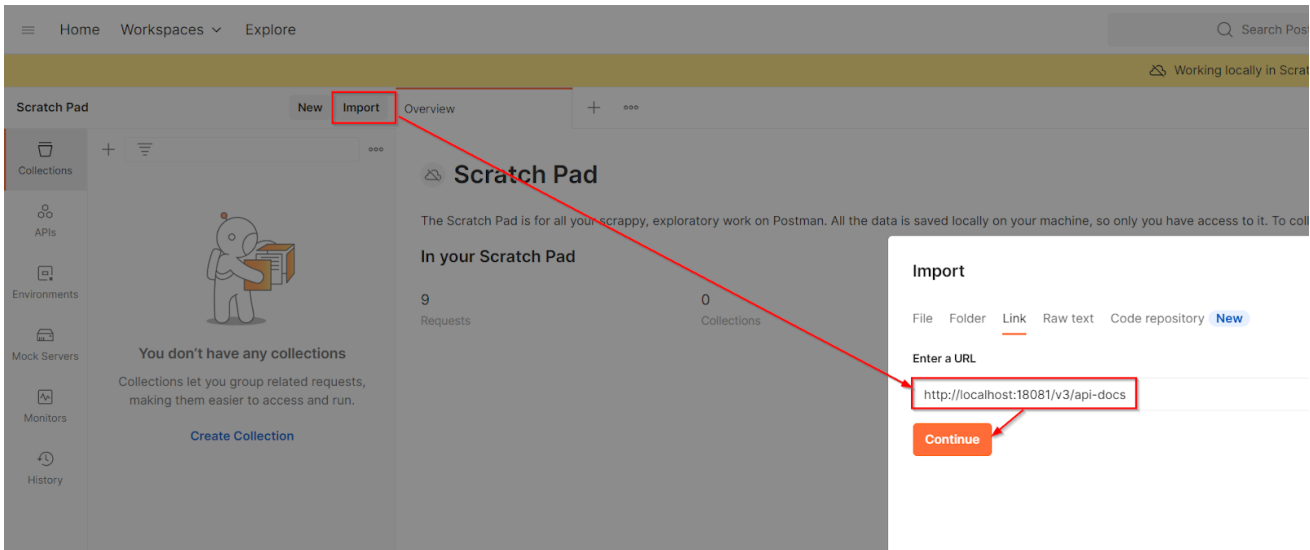
Un cop feta la implementació, es pot obtenir el "contracte" o la definició de cadascun dels serveis en format JSON a l'adreça v3/api-docs:

- <http://localhost:18081/v3/api-docs> (productCatalog)
- <http://localhost:18082/v3/api-docs> (user)

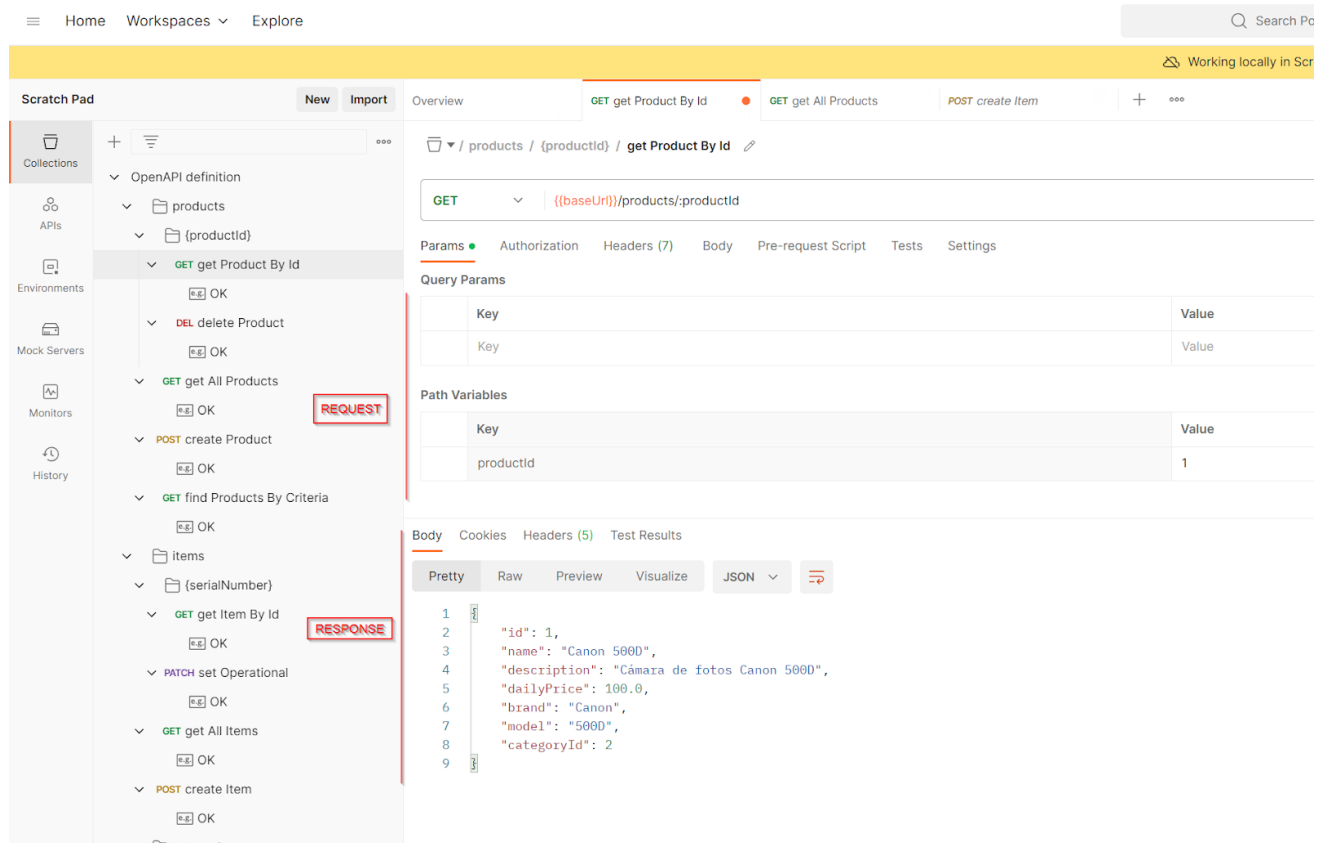
```
{
  "openapi": "3.0.1",
  "info": {
    "title": "Swagger definition",
    "version": "v1",
    "servers": [
      {
        "url": "http://localhost:8081",
        "description": "Generated server url"
      }
    ],
    "paths": {
      "/products": {
        "get": {
          "tags": [
            "product-rest-controller"
          ],
          "operationId": "getAllProducts",
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "application/json": {
                  "schema": {
                    "type": "array",
                    "items": {
                      "type": "object",
                      "properties": {
                        "id": {
                          "type": "integer",
                          "format": "int64"
                        },
                        "name": {
                          "type": "string"
                        },
                        "description": {
                          "type": "string"
                        },
                        "dailyPrice": {
                          "type": "number",
                          "format": "double"
                        },
                        "brand": {
                          "type": "string"
                        },
                        "category": {
                          "type": "string"
                        },
                        "parentCategory": {
                          "type": "string"
                        },
                        "product": {
                          "type": "object",
                          "properties": {
                            "id": {
                              "type": "integer",
                              "format": "int64"
                            },
                            "name": {
                              "type": "string"
                            },
                            "description": {
                              "type": "string"
                            },
                            "dailyPrice": {
                              "type": "number",
                              "format": "double"
                            },
                            "brand": {
                              "type": "string"
                            },
                            "category": {
                              "type": "string"
                            },
                            "parentCategory": {
                              "type": "string"
                            }
                          }
                        }
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

El microservei notificació no exposa cap operació i, per tant, encara que es pugui accedir a la seva interfície swaggerUI, ofereix un contracte buit.

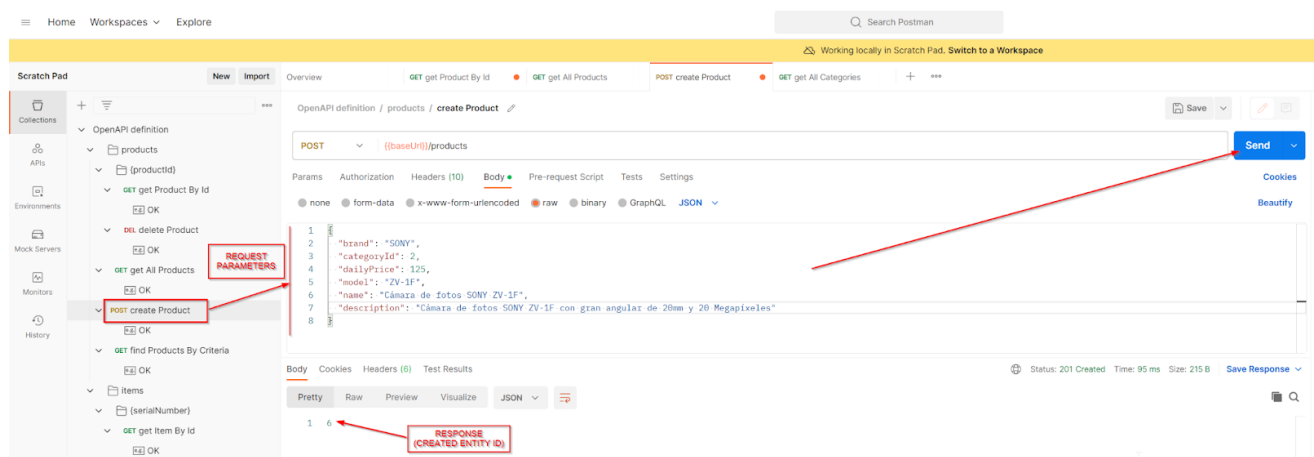
Aquest fitxer JSON es pot descarregar o importar directament a una eina externa com Postman, des d'on serà molt més senzill realitzar totes les proves necessàries per verificar el funcionament correcte dels diferents serveis.



Des d'aquesta vista podrem realitzar qualsevol crida a les APIs des d'un entorn gràfic i consultar els valors de retorn.



The screenshot shows the Postman interface with a workspace named 'Working locally in Scratch Pad'. The left sidebar shows a collection of APIs under 'OpenAPI definition' and 'products'. The main panel displays a GET request to 'get Product By Id' with the URL '({baseUri})/products/productId'. The 'Params' tab is selected, showing a single parameter 'productid' with a value of '1'. The 'Body' tab is also visible, showing a JSON response with details for a Canon 500D camera. Red boxes highlight the 'REQUEST' and 'RESPONSE' sections.



The screenshot shows the Postman interface with a workspace named 'Working locally in Scratch Pad'. The left sidebar shows a collection of APIs under 'OpenAPI definition' and 'products'. The main panel displays a POST request to 'create Product' with the URL '({baseUri})/products'. The 'Body' tab is selected, showing a JSON request body with details for a Sony ZV-1F camera. The 'Send' button is highlighted with a red arrow. The 'Body' tab is also visible, showing a JSON response with details for a Sony ZV-1F camera. Red boxes highlight the 'REQUEST PARAMETERS' and 'RESPONSE (CREATED ENTITY ID)' sections.

Exercici 2 (2 punts)

L'èxitós llançament del projecte **Photo&Film4You** ha despertat l'interès d'**IndieFilmify**, una *startup* emergent especialitzada en la gestió de produccions cinematogràfiques independents. **IndieFilmify** ofereix eines per a la planificació de rodatges o contractació de personal, i ara vol integrar serveis de lloguer d'equips directament en el seu sistema.

IndieFilmify vol integrar la seva plataforma amb la de **Photo&Film4You**, per oferir als seus clients la possibilitat de reservar i gestionar lloguers d'equipament directament des del seu sistema. En cas d'arribar a un acord, això representaria una oportunitat per aconseguir una audiència més àmplia dins del sector cinematogràfic independent, a canvi d'una petita comissió per cada transacció generada a través d'**IndieFilmify**.

La dinàmica de funcionament implicaria que, al gestionar un rodatge dins d'**IndieFilmify**, els usuaris podran accedir a una secció de lloguer d'equipaments. Des d'aquí, **IndieFilmify** consultarà en temps real el catàleg de **Photo&Film4You** (junt amb d'altre proveïdors) i mostrarà els productes disponibles. Si un usuari mostra interès per algun dels models disponibles per llogar a **Photo&Film4You**, la reserva es realitzarà directament des d'**IndieFilmify**, amb la gestió del pagament i l'actualització de l'inventari realitzats en la nostra plataforma de manera automàtica.

Malgrat la direcció considera favorablement aquesta proposta, el Director de Tecnologia (CTO) té algunes reticències. En una primera reunió tècnica amb el CTO d'**IndieFilmify**, s'ha pogut observar que es tracta d'una aplicació basada en serveis que necessitarà realitzar consultes al nostre catàleg de productes, reserves i gestió de lloguers d'equipaments de **Photo&Film4You**, i durant les primeres proves d'integració s'ha detectat problemes crítics: lloguers cancel·lats no sempre alliberen l'estoc, pagaments confirmats sense disponibilitat real de la unitat de producte, o notificacions de disponibilitat que no sempre arriben als clients que han sol·licitat les alertes. Aquests problemes generen inconsistències en les plataformes, afectant la confiabilitat del servei i l'experiència d'usuari.

Considerant aquest escenari, es demana escollir i justificar l'ús d'un patró per resoldre la gestió de transaccions distribuïdes de la nostra plataforma segons les condicions acordades, i quins són els pros/cons de la vostra decisió.

Lliurables

La resposta a aquest exercici es lliurarà en format text PDF.

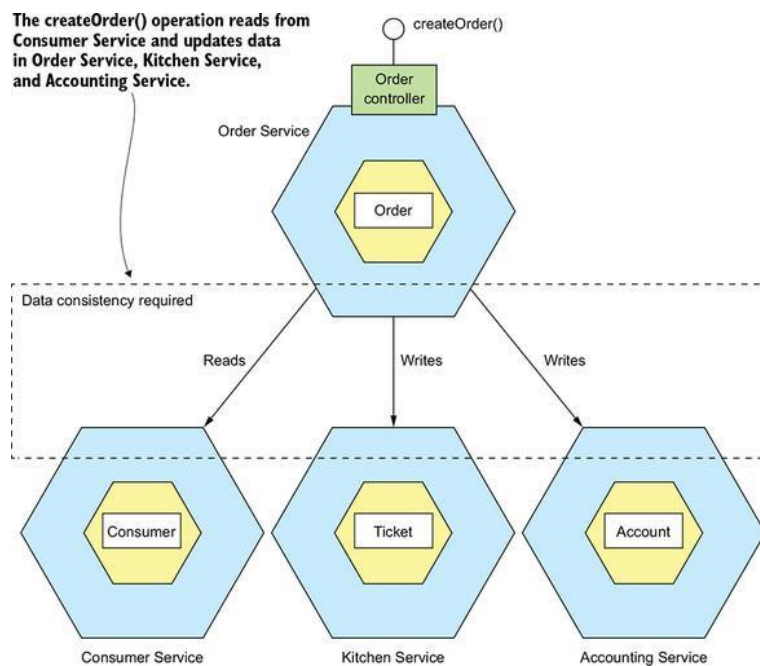
Solució

El patró escollit per resoldre la problemàtica plantejada és SAGA:

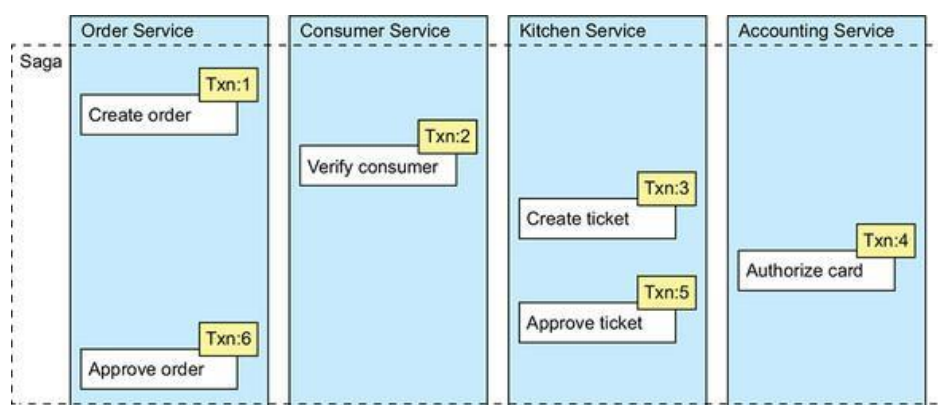
<http://microservices.io/patterns/data/saga.html>

El patró SAGA ens ajuda a mantenir coherència entre serveis i dades, i consisteix en una seqüència de transaccions locals coordinades per actualitzar de forma fiable les dades disperses en múltiples microserveis. Al processar una petició, el servei lliura missatges asíncrons a altres serveis, garantint un acoblament flexible entre aquests.

Una *API Gateway* és una passarel·la que serveix com a punt únic d'accés a una aplicació des de l'exterior. L'aplicació d'aquest patró ens permet oferir una façana a l'exterior que encapsula tota l'arquitectura interna de manera similar al que es fa aplicant el patró *Façana*, bastant conegut dins de la programació orientada a objectes.

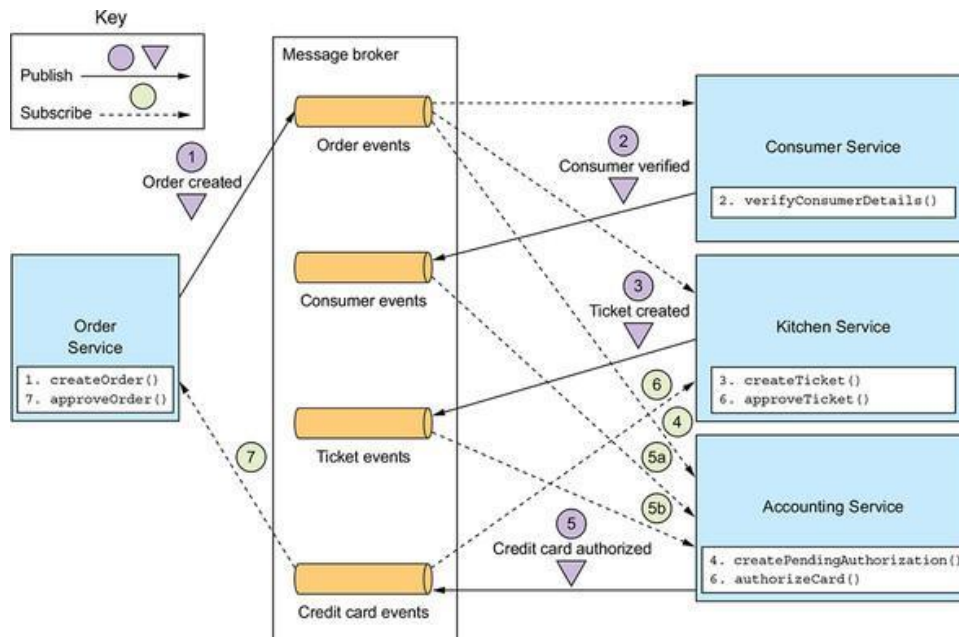


Cada microservei disposa la seva base de dades, i hem de garantir la coherència de les dades. Per tant, en l'operació del sistema la finalització d'una transacció desencadena l'execució de la següent.



Existeixen dues maneres diferents d'estructurar la lògica de coordinació: *Choreography* o *Orchestration*. Cada opció té els seus pros i contres, i s'acceptaran sempre que es justifiquin adequadament. En aquesta solució presentem la justificació d'escollir el patró SAGA amb la lògica de **Choreography**, recomanada per les SAGAs més simples (en altre cas, es recomana *Orchestration*).

Aquest patró distribueix entre tots els participants de la SAGA la presa de decisions i seqüenciació, principalment la comunicació és per intercanvi d'esdeveniments:



En el cas d'aquest exemple de la imatge, el flux seria el següent:

- Es genera una ordre i es publica un esdeveniment inicial, en estat pendent d'aprovació.
- Es verifiquen els detalls des del consumidor, i es publica un nou esdeveniment
- Es valida l'ordre, es crea un ticket en estat pendent, i es torna a publicar un nou esdeveniment.
- Es crea una autorització del pagament després de validar la comanda, i es processa el cobrament i es publica un darrer esdeveniment.
- Finalment, si el pagament és autoritzat, s'actualitzen els estats de l'ordre i els tiquets associats a l'estat acceptat/aprovat.

La missatgeria asíncrona garanteix que s'executen tots els passos d'una SAGA, inclòs si un o més serveis no es troben temporalment disponibles.

A més, existeixen altres avantatges:

- **Simplicitat:** els serveis publiquen esdeveniments quan creen, actualitzen o modifiquen dades, sense necessitat d'un orquestrador central.

- **Baix acoblament:** els serveis subscrits a esdeveniments reaccionen a aquests sense dependre directament uns dels altres.

Els principals inconvenients:

- La lògica està distribuïda entre serveis, per tant és més difícil d'entendre i depurar
- Dependències cícliques: els serveis responen a esdeveniments entre si, complicant l'arquitectura.
- Manca d'aïllament entre SAGAs: una SAGA pot canviar unes dades mentre una altra SAGA està llegint aquestes abans de completar les actualitzacions.
- Complexitat per revertir canvis davant un error. Les aplicacions han de permetre operacions per desfer canvis confirmats en passos anteriors: són transaccions de compensació,

Podem utilitzar contramesures o patrons per paliar aquestes anomalies com *Lost updates*, *dirty reads*, o *fuzzy/nonrepeteable reads*.

Per exemple el patró Circuit Breaker prevé la propagació d'errors aturant temporalment les sol·licituds a serveis fallits. També cal garantir la idempotència entre les operacions, garantitzant que aquestes es puguin repetir sense causar respostes diferents, afegir telemetria o observabilitat que facilitin la detecció d'errors o de problemes de rendiment, mitjançant la monitorització en temps real del sistema.

Referències:

- Microservices Patterns (Richardson). Chapter 4: "Managing Transactions With Sagas".
- Pattern: Saga: <https://microservices.io>

Recursos

Recursos bàsics

- "Microservices Patterns" (Chris Richardson): Chapter 6: "Developing business logic with event sourcing" (excepte secció 6.3).
- "Microservices Patterns" (Richardson): Chapter 7: "Implementing queries in a microservice architecture".
- "Microservices Patterns" (Richardson): Chapter 8: "External API patterns" (excepte secció 8.3).

Recursos complementaris

- Repositori de codi font de l'exemple del llibre Microservices Patterns: <https://github.com/microservices-patterns/ftgo-application>
- Exemple de programació de microserveis: <https://github.com/vmudigal/microservices-sample>

Criteris d'avaluació

- En aquesta activitat no està permès utilitzar eines d'intel·ligència artificial, i cal realitzar-la de manera **estrictament individual**. En cas de detectar còpies o qualsevol altra forma de frau acadèmic, es penalitzarà la prova amb una D com a nota. Això inclou la reutilització de solucions d'aquesta pràctica de semestres anteriors. Al pla docent i al [web sobre integritat acadèmica i plagi de la UOC](#) trobareu informació sobre què es considera conducta irregular en l'avaluació i les conseqüències que pot tenir.
- El pes de cada exercici està indicat en l'enunciat.
- Trobareu una descripció ampliada dels criteris d'avaluació de cada exercici en l'annex.
- Cal justificar la solució a cadascun dels exercicis. Es valorarà tant la correcció de la solució com la justificació donada.

Format i data de lliurament

Cal lliurar un únic arxiu ZIP amb les respostes a tots els exercicis. El nom de l'arxiu ha de ser: *CognomsNom_EPCSD_PRAC2.zip* amb la següent estructura:

- ZIP amb el lliurament de la primera part de l'exercici 1, que ha de contenir:
 - ZIP amb el codi font
 - un arxiu JAR executable per cada microservei implementat
 - un arxiu readme.txt amb instruccions addicionals per arrencar la infraestructura i/o microserveis (sobretot en cas de canvis en la configuració o en el docker-compose)
- Captures de pantalla de Postman per la segona part de l'exercici 1.
- PDF de resposta de l'exercici 2.

Aquest document es lliurarà a l'espai "*Lliurament de la PRAC2*" de l'aula, abans de les **23:59 hores del dia 19 de maig de 2025**. No s'acceptaran lliuraments fora de termini.

Annex

Criteris d'avaluació

	Excel·lent (10)	Notable (7)	Suficient (5)	Insuficient (2,5)	Sense resposta / tot malament (0)
Ex. 1 1a part	• Nombre d'operacions no presents a l'esquelet implementades correctament des del punt de vista funcional (amb 10 màxima puntuació) • Ús correcte dels verbs HTTP • Control d'errors (IDs que no existeixen, errors de tipus, etc.), validacions i retorn de l'estat HTTP adequat al resultat de la crida • Col·laboració completa entre serveis per crides REST • Utilitzar tots els paràmetres query/path segons les convencions estàndard • Utilitzar DTOs per agrupar inputs/outputs • Justificació/explicació correcta a les parts on cal. • Qualitat màxima de la implementació (llegibilitat, formatat, sense warnings, etc.) (6)	• Nombre d'operacions no presents a l'esquelet implementades correctament des del punt de vista funcional (mínim 8) • Ús en general correcte dels verbs HTTP • Control d'errors (IDs que no existeixen, errors de tipus, etc.), validacions i retorn de l'estat HTTP en general adequat al resultat de la crida • Alguns problemes amb la col·laboració entre serveis per crides REST • Utilitzar la majoria de paràmetres query/path segons les convencions estàndard • Utilitzar DTOs per agrupar inputs/outputs • Justificació/explicació incorrecta o en llocs on no cal. • Qualitat general acceptable de la implementació (llegibilitat, formatat, sense warnings, etc.) (4)	• Nombre d'operacions no presents a l'esquelet implementades correctament des del punt de vista funcional (mínim 5) • Ús en general incorrecte dels verbs HTTP • Control d'errors (IDs que no existeixen, errors de tipus, etc.), validacions i retorn de l'estat HTTP en general inadequat al resultat de la crida • Moltes problemes amb la col·laboració entre serveis per crides REST • Utilitzar paràmetres query/path, però no segueixen les convencions estàndard • No utilitzar DTOs per agrupar inputs/outputs • Absència de justificació/explicació. • Qualitat general pobre de la implementació (llegibilitat,	• Nombre d'operacions no presents a l'esquelet implementades correctament des del punt de vista funcional (menys de 5) • Ús incorrecte dels verbs HTTP • Control d'errors (IDs que no existeixen, errors de tipus, etc.), validacions i retorn de l'estat HTTP no adequat al resultat de la crida • No hi ha col·laboració entre serveis per crides REST • No utilitzar paràmetres query/path i/o no segueixen les convencions estàndard • No utilitzar DTOs • Qualitat inacceptable de la implementació (llegibilitat, formatat, sense warnings, etc.) (1,5)	No resposta, no hi ha cap justificació o tot malament (0)

			formatat, sense warnings, etc.) (3)		
Ex. 1 2a part	Evidències per totes les operacions correctes, amb informació (entrada i sortida) concreta per cada una d'elles (2)	Evidències per aproximadament 3/4 de les operacions correctes, amb informació (entrada i sortida) concreta per cada una d'elles. (1,5)	Evidències per aproximadament la meitat de les operacions correctes, amb informació (entrada i sortida) concreta per cada una d'elles. (1)	Evidències per aproximadament menys de la meitat de les operacions correctes, amb informació (entrada i sortida) concreta per cada una d'elles. (0,5)	No resposta, no hi ha cap operació addicional respecte les que ja tenia l'esquelet o bé no es pot importar l'arxiu de definició (0)
Ej. 2	Elecció correcta del patró i justificació completa i correcte que fa que aquesta elecció sigui la més adequada (2)	Elecció correcta del patró, però justificació incorrecta de que aquesta elecció sigui la més adequada (1,5)	Elecció incorrecta del patró, però justificació acceptable que fa que aquesta elecció sigui la més adequada (1)	Elecció incorrecta del patró i sense justificació o incorrecte que fa que aquesta elecció sigui la més adequada (0,5)	No resposta, no hi ha cap elecció ni justificació (0)